

Task 10 - Part 02: Research Topics

Parallel Programming & Concurrency, Unit Testing / TDD, Async Programming

2. Parallel Programming and Concurrency

Concurrency is the ability of a program to deal with multiple tasks that are in progress at the same time - conceptually overlapping in time, even if they aren't literally executing at the exact same instant (e.g., a single CPU core rapidly switching between tasks). **Parallelism** is a stronger, more specific case: tasks are actually running **simultaneously**, typically across multiple CPU cores. Concurrency is about structuring a program to handle many things at once; parallelism is about actually executing many things at the exact same time.

In C# and .NET, this is supported through tools such as:

- **Task Parallel Library (TPL)** - `Parallel.For`, `Parallel.ForEach`, and the `Task` class, which make it easy to split work across multiple threads.
- **Threads and the `Thread` class** - lower-level, manual control over separate execution paths (rarely used directly today in favor of higher-level abstractions).
- **PLINQ (Parallel LINQ)** - a parallel version of LINQ that automatically distributes query execution across multiple cores.
- **Synchronization primitives** - locks (lock keyword), `Mutex`, `Semaphore`, and `Monitor`, used to safely coordinate access to shared data and avoid race conditions when multiple threads work concurrently.

The main challenges in this area are avoiding **race conditions** (when multiple threads read/write shared data unpredictably), **deadlocks** (when threads wait on each other indefinitely), and correctly reasoning about thread safety, all in exchange for the significant performance benefits parallelism can offer on modern multi-core hardware for CPU-intensive workloads.

2. Unit Testing and Test-Driven Development (TDD)

Unit testing means writing small, automated tests that verify an individual unit of code (typically a single method or class) behaves correctly in isolation, independent of the rest of the system. In C#, this is commonly done with frameworks like **xUnit**, **NUnit**, or **MSTest**, often paired with mocking libraries (like `Moq`) to isolate the unit under test from its real dependencies (databases, external services, etc.).

Test-Driven Development (TDD) is a development workflow built around writing tests **before** writing the actual implementation code, typically following a "Red-Green-Refactor" cycle:

- **Red** - write a failing test that describes the behavior you want.
- **Green** - write the minimum amount of implementation code needed to make that test pass.
- **Refactor** - clean up and improve the code (and/or the test) while keeping all tests passing.

The benefit of this approach is that it forces requirements to be clearly defined upfront as testable behavior, tends to produce more modular and loosely coupled code (since testable code is naturally structured that way), and leaves behind a comprehensive automated test suite that catches regressions whenever the code changes later.

2. Asynchronous Programming with async and await

Asynchronous programming lets a program start a long-running operation (like a network call, file I/O, or database query) **without blocking the calling thread** while it waits for that operation to complete - the thread is freed up to do other work in the meantime, and execution resumes once the operation finishes. In C#, this is expressed using the **async** and **await** keywords together with the **Task** and **Task<T>** types:

- A method marked **async** can use **await** inside it to pause execution at an asynchronous operation (e.g., `await httpClient.GetStringAsync(url)`) without blocking the thread - control returns to the caller until the awaited operation completes.
- An async method typically returns **Task** (for no result) or **Task<T>** (for a result of type T), letting callers await it in turn, or handle it as a Task like any other asynchronous unit of work.
- This is especially important for I/O-bound operations (web requests, database access, file reads/writes), where the thread would otherwise sit idle waiting for a response - async keeps the application responsive (crucial in UI apps) and scalable (crucial in server/web apps handling many simultaneous requests) by not tying up threads unnecessarily.

It's worth distinguishing this from parallelism/concurrency above: async/await is primarily about efficiently handling **waiting** (I/O-bound work) without wasting threads, whereas the Task Parallel Library is primarily about running **CPU-bound** work across multiple cores simultaneously - though both are built on top of the same underlying Task infrastructure in .NET.